

useEffect — A Practical Guide

A deep-dive covering the two components in this project ([LiveClock.tsx](#) and [TitleCounter.tsx](#)), the mechanics of the dependency array, cleanup functions, and the bugs that appear when you get them wrong.

Table of contents

1. What `useEffect` is for
 2. Anatomy of the hook
 3. The dependency array — three modes
 4. Cleanup functions
 5. Case study 1 — `TitleCounter`
 6. Case study 2 — `LiveClock`
 7. Bug lab — what breaks when you get it wrong
 8. React 18 StrictMode
 9. Rules of thumb
-

What `useEffect` is for

Render functions in React must be **pure** — same inputs, same output, no side effects. But real apps need side effects: setting `document.title`, starting timers, subscribing to events, fetching data, etc.

`useEffect` is the escape hatch. It lets you run code **after** React commits the render to the DOM, in a controlled and reversible way.

Think of `useEffect` as: *"After you paint this render to the screen, do this thing. And here's how to undo it later."*

Anatomy of the hook

```
useEffect(  
  () => {  
    // 1. Setup — runs after commit  
    //   Do the side effect here.  
  
    return () => {
```

```

    // 2. Cleanup – runs before the next setup, and on unmount
    //   Reverse whatever setup did.
  };
},
[/* 3. Dependencies */]
);

```

- **Setup function** — the effect itself.
- **Cleanup function** — optional; returned from setup. React calls it before re-running the effect and when the component unmounts.
- **Dependency array** — controls *when* the effect re-runs.

The dependency array — three modes

Second argument	When the effect runs	Typical use
<code>[]</code>	Once on mount, cleanup on unmount	Timers, subscriptions, event listeners that don't depend on props/state
<code>[a, b]</code>	On mount + whenever <code>a</code> or <code>b</code> changes	Syncing with a specific value (like <code>document.title</code> with <code>count</code>)
<i>(omitted)</i>	After every render	Almost always a bug when combined with side effects

Mental model

React compares the new dependency array to the previous one using `Object.is` element-by-element. If any element differs, cleanup + setup run again.

```

useEffect(() => { /* ... */ }, [count]);
//               ^^^^^
// React remembers the previous [count]. On each render,
// it checks: did count change? If yes, re-run.

```

Cleanup functions

The cleanup function is your promise to React: *"I created something ongoing — I'll tear it down before you re-run me or unmount me."*

You need cleanup for

- `setInterval` → `clearInterval`
- `setTimeout` → `clearTimeout`
- `addEventListener` → `removeEventListener`
- WebSocket / EventSource → `.close()`
- Subscriptions (Redux, RxJS, Firebase) → unsubscribe function
- `AbortController` for fetch → `controller.abort()`

You don't need cleanup for

- Setting `document.title` (idempotent overwrite)
- Logging
- Updating a ref
- One-shot DOM mutations that don't accumulate

Case study 1 — TitleCounter

```
import { useEffect, useState } from "react";

export default function TitleCounter() {
  const [count, setCount] = useState<number>(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={() => setCount((c) => c + 1)}>Increment</button>
    </>
  );
}
```

Why `[count]` and not `[]` ?

The effect **reads** `count`. If we used `[]`, the effect would only run on mount and always show `Count: 0` in the tab title, even after 100 clicks. Listing `count` as a dependency tells React: *"re-sync the title whenever count changes."*

Why no cleanup?

Setting `document.title = "..."` is idempotent. Each new render simply overwrites the previous value — nothing accumulates, nothing leaks. There's nothing to tear down.

Why the functional updater `setCount((c) => c + 1)` ?

It reads the latest count from React's internal state, avoiding stale-closure bugs when multiple updates queue up. Not strictly required here, but a good habit.

Case study 2 — `LiveClock`

```
import { useState, useEffect } from "react";

export default function LiveClock() {
  const [time, setTime] = useState<Date>(new Date());

  useEffect(() => {
    const intervalId = setInterval(() => {
      setTime(new Date());
    }, 1000);

    return () => {
      clearInterval(intervalId);
    };
  }, []);

  return <h2>{time.toLocaleTimeString()}</h2>;
}
```

Why `[]` ?

`setInterval` is a **one-time setup**. We want exactly one active timer for the component's lifetime. An empty deps array means: *"set this up on mount, don't re-run it, tear it down on unmount."*

Why cleanup with `clearInterval` ?

The interval keeps firing until explicitly cleared. Without cleanup, the timer would outlive the component and cause the bugs described in the next section.

Why `setTime(new Date())` doesn't need `time` as a dependency

The callback doesn't read `time` — it just writes a fresh `Date`. No stale closure, no missing dependency.

Bug lab — what breaks when you get it wrong

Bug 1 — Removing `clearInterval` from `LiveClock`

```
useEffect(() => {
  setInterval(() => setTime(new Date()), 1000);
  // ❌ no cleanup
}, []);
```

What happens:

- The interval keeps running after unmount → **memory leak**.
- React logs "Can't perform a state update on an unmounted component" (older versions).
- In **StrictMode** dev mode, the effect runs twice → **two intervals** stack up, and **setTime** fires twice per second.
- Every route change / conditional unmount adds another orphaned interval.

Bug 2 — Removing the `[]` from **LiveClock**

```
useEffect(() => {
  const intervalId = setInterval(() => setTime(new Date()), 1000);
  return () => clearInterval(intervalId);
}); // ❌ no deps array – runs after every render
```

What happens:

Because cleanup exists, only one interval is active at a time. But the interval is destroyed and recreated on **every render**, and the 1-second countdown restarts each time.

Trace when a parent re-renders faster than once per second:

```
t=0.0s  Mount → Interval A scheduled for t=1.0s
t=0.3s  Parent re-renders → clear A, Interval B scheduled for t=1.3s
t=0.6s  Parent re-renders → clear B, Interval C scheduled for t=1.6s
t=0.9s  Parent re-renders → clear C, Interval D scheduled for t=1.9s
```

The clock freezes. No interval ever survives long enough to fire.

Bug 3 — Missing dependency in **TitleCounter**

```
useEffect(() => {
  document.title = `Count: ${count}`;
}, []); // ❌ count is read but not listed
```

What happens:

The effect captures `count = 0` in its closure. On mount, the title becomes `"Count: 0"`. Clicking the button updates state and re-renders, but the effect never re-runs — the title is permanently stuck at

"Count: 0" . This is the classic **stale closure** bug.

The `react-hooks/exhaustive-deps` ESLint rule catches this.

Bug 4 — Object/array dependencies causing infinite loops

```
useEffect(() => {  
  fetchData(options);  
}, [{ id: 42 }]); // ❌ new object literal every render
```

`{ id: 42 } !== { id: 42 }` by reference, so React thinks the dependency changed on every render.
Effect re-runs → new fetch → state update → re-render → new object → ∞.

Fix: depend on primitives (`[id]`) or memoize the object with `useMemo` .

React 18 StrictMode

In development, `<React.StrictMode>` intentionally does this to every effect on mount:

```
setup → cleanup → setup
```

The double-invoke helps you catch missing cleanup. If your effect leaks (like Bug 1), StrictMode makes the leak immediately visible — you'll see two intervals firing.

This does not happen in production. Don't disable StrictMode to "fix" the double log — fix the missing cleanup instead.

Rules of thumb

1. **Every reactive value read inside the effect belongs in the deps array.** Let `react-hooks/exhaustive-deps` enforce this.
2. **If you set up something ongoing, return a cleanup that tears it down.** Intervals, timeouts, listeners, subscriptions, connections.
3. **Use `[]` only when the effect truly has no reactive dependencies.** If you're using `[]` to "run once", double-check that you're not silently capturing stale values.
4. **Never omit the dependency array entirely** unless you deliberately want to run on every render (rare, almost always wrong for side effects).
5. **Prefer primitives in deps.** For objects/arrays, memoize them or destructure into primitives.
6. **Don't disable StrictMode to silence warnings.** Fix the underlying leak.

Quick reference table

Scenario	Deps	Cleanup?
Set <code>document.title</code> from state	<code>[state]</code>	No
Start an interval	<code>[]</code>	<code>clearInterval</code>
Subscribe to a WebSocket URL that can change	<code>[url]</code>	<code>socket.close()</code>
Fetch data for a route param	<code>[id]</code>	<code>controller.abort()</code>
Attach a global keydown listener	<code>[]</code>	<code>removeEventListener</code>
Log every render (debug)	<code>omit</code>	No